



Pontifícia
Universidade
Católica do
Rio de Janeiro

INF1304 – Distribuição e Concorrência

Trabalho 1: Balanceamento de Carga, Elasticidade e Failover com Kafka em Kubernetes

Alunos:

2310822 Eduardo Eugênio de
Souza
2210508 Gabriel Augusto Gedey
2311271 Gabriel Martins Mendes

Professor:

Alexandre Malheiros Meslin

Rio de Janeiro,
Setembro de 2026

SUMÁRIO

1. INTRODUÇÃO	5
2. ARQUITETURA	5
2.1. Visão geral	5
2.2. Cluster Kafka	6
2.3. Sensores (produtores)	7
2.4. Consumidores (processadores de dados)	8
2.5. Controlador	9
2.6. Armazenamento e configuração	9
3. INSTALAÇÃO E USO	10
3.1. Pré-requisitos	10
3.2. Instalação	10
3.3. Operação	10
4. TESTES DE FALHA E ELASTICIDADE	11
4.1. Falha de um broker Kafka	12
4.2. Falha de um consumidor e rebalanceio	12
4.3. Elasticidade	12
5. RESULTADOS	13
5.1. O que funcionou	13
5.2. O que não funcionou	13
6. CONCLUSÃO	14
REFERÊNCIAS	15
A Logs de execução	16
AA Failover do broker	16
AB Rebalanceamento de consumidores	16
AC Elasticidade	16

LISTA DE FIGURAS

Figura 1 Arquitetura do sistema: fluxo de dados (linha central) e malha de controle (linha inferior).	6
---	---

LISTA DE TABELAS

Tabela 1 Componentes do sistema e recursos Kubernetes correspondentes.	6
Tabela 2 Intervalo entre medições de cada tipo, por máquina.	8
Tabela 3 Principais comandos de operação do Makefile.	11

1. INTRODUÇÃO

Uma fábrica inteligente possui diversas máquinas espalhadas em setores como linha de produção, refrigeração e empacotamento. Cada máquina é equipada com sensores que enviam continuamente medições de temperatura, vibração, consumo de energia e emissão de CO₂ para um sistema central, que as processa em tempo real para detectar anomalias, falhas ou padrões de uso.

A equipe de engenharia dessa fábrica exige três propriedades do sistema central: ser tolerante a falhas, isto é, não parar quando uma de suas partes cair; escalar com facilidade conforme o número de sensores aumenta; e balancear a carga de forma eficiente entre os processadores de dados. Este trabalho apresenta uma implementação desse sistema com o Apache Kafka (Apache Software Foundation, [S.d.]) executado em um cluster Kubernetes (The Kubernetes Authors, [S.d.]), e demonstra o balanceamento de carga, a elasticidade e o *failover* por meio de testes de falha.

O objetivo é construir e validar um cluster Kafka com múltiplos brokers e tópico particionado, sensores simulados como produtores em containers distintos, consumidores organizados em um grupo de consumo com balanceamento automático de partições, e mecanismos para simular a queda de um broker e de um consumidor, registrando o comportamento do sistema por meio de logs.

Este relatório está organizado da seguinte forma. O capítulo de arquitetura descreve os componentes e como eles se comunicam. O capítulo de instalação e uso explica como implantar e operar a aplicação. O capítulo de testes de falha e elasticidade descreve os procedimentos executados, e o capítulo de resultados apresenta o que funcionou e o que não funcionou.

2. ARQUITETURA

2.1. Visão geral

O sistema é composto por cinco tipos de componentes, todos executados como pods em um cluster Kubernetes (K3s) e configurados por variáveis de ambiente. Os sensores publicam medições em formato JSON no tópico `dados-sensores` do Kafka. Os consumidores, que pertencem a um mesmo grupo de consumo, leem esse tópico, comparam cada medição com limites predefinidos e gravam as leituras e os alertas em um banco PostgreSQL. Os consumidores também registram os alertas de cada máquina em um Redis compartilhado. Quando uma máquina acumula alertas em excesso, publicam um comando no tópico `comandos-fabrica`, lido pelo controlador, que remove o pod do sensor correspondente por meio da API do Kubernetes; o Deployment do sensor então recria o pod, fechando o laço mostrado na figura. A Figura 1 resume esse fluxo e a Tabela 1 lista os componentes.

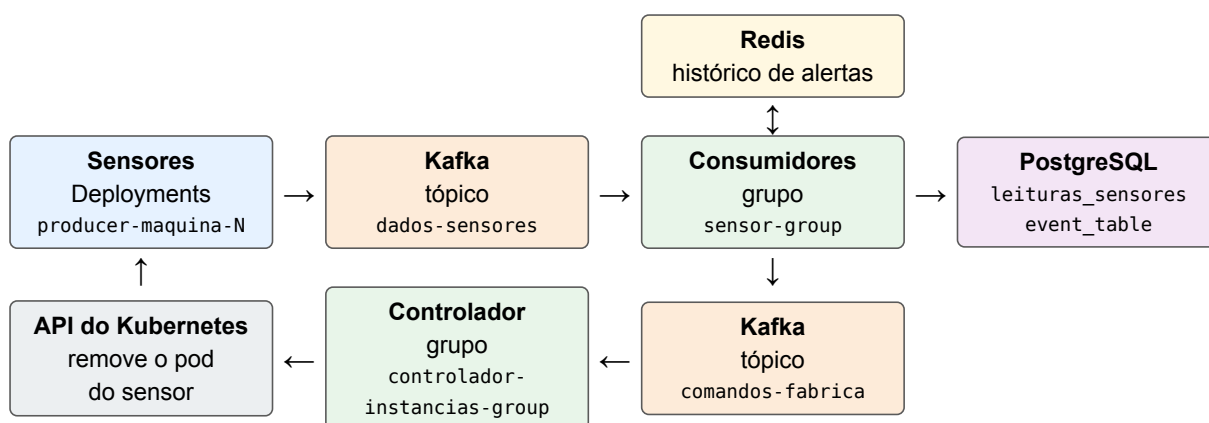


Figura 1: Arquitetura do sistema: fluxo de dados (linha central) e malha de controle (linha inferior).

Componente	Recurso Kubernetes	Réplicas	Função
Brokers Kafka	StatefulSet kafka	três	Armazenam e distribuem as mensagens dos tópicos
Sensores	Deployments producer-maquina-N (3 no início)	1 cada	Geram medições periódicas e as publicam em dados-sensores
Consumidores	Deployment consumer	3	Processam as medições, detectam anomalias e gravam no banco
Consumidores extras	Deployments consumer-extra-N	0 no início	Criados sob demanda ao escalar o grupo além de três consumidores; processam mais rápido
Controlador	Deployment controlador	2	Remove pods de máquinas problemáticas a pedido dos consumidores
PostgreSQL	StatefulSet postgres	1	Guarda leituras, alertas e eventos de auditoria
Redis	Deployment redis	1	Compartilha entre os consumidores o histórico de alertas por máquina
Adminer	Deployment adminer	1	Interface web para consultar o banco

Tabela 1: Componentes do sistema e recursos Kubernetes correspondentes.

2.2. Cluster Kafka

O cluster Kafka é implantado como um StatefulSet que cria três pods, um broker em cada, com nomes estáveis e ordenados (`kafka-0`, `kafka-1`, ...). Essa identidade fixa é necessária porque cada broker precisa ser localizável pelos demais e pelos clientes. Um Service headless (`kafka-headless`) fornece a resolução de nomes por pod, e um Service comum (`kafka`, porta 9092) serve de ponto de entrada para produtores e consumidores.

O cluster opera em modo KRaft, sem Zookeeper: cada pod exerce ao mesmo tempo os papéis de broker e de controller, e os controllers formam um quórum que mantém os metadados do cluster e conduz a eleição de líderes de partição. A configuração de cada broker é gerada na inicialização por um script (`setup-kraft.sh`),

armazenado em um ConfigMap. O script deriva o identificador do nó a partir do nome do pod (`kafka-0` recebe o identificador 0), monta o arquivo de configuração com os listeners de cliente (porta 9092) e de controller (porta 9093) e formata o armazenamento com o identificador do cluster. Esse identificador, um UUID compartilhado por todos os brokers, é lido da variável `KAFKA_CLUSTER_ID`, definida em um ConfigMap; ele precisa ser o mesmo em todos os nós e só deve mudar quando o cluster é recriado do zero.

Os tópicos `dados-sensores` e `comandos-fabrica` são criados por um Job do Kubernetes (`kafka-init-topics`) logo após a subida dos brokers, cada um com doze partições e fator de replicação 3. A criação automática de tópicos pelos brokers está desativada, para que os tópicos sempre tenham a configuração definida no Job, e o Job apaga e recria os dois tópicos a cada implantação, de modo que cada execução começa com os tópicos vazios. Os sensores e os consumidores só iniciam depois que o tópico `dados-sensores` existe: cada pod tem um contêiner de inicialização que aguarda o tópico aparecer na listagem do Kafka.

As partições permitem que a leitura de um tópico seja dividida entre vários consumidores do grupo. A replicação é o que permite tolerar a queda de um broker: com fator de replicação 3, cada partição tem uma cópia em cada um dos três brokers, sendo uma delas a líder, que atende produtores e consumidores, e as demais seguidoras, que se mantêm sincronizadas. Se o broker que lidera uma partição cai, o controller elege como nova líder uma das réplicas em sincronia que restaram. O quórum de controllers também tolera a perda de um nó, porque com três votantes a maioria necessária é de dois. Os tópicos internos do Kafka, como o que guarda as posições de leitura dos grupos de consumo, também usam fator de replicação 3, com no mínimo duas réplicas em sincronia.

Os brokers não têm volume persistente: os dados ficam no sistema de arquivos do contêiner (`/tmp/kraft-logs`). Quando o pod de um broker é recriado, ele volta sem dados e precisa copiar das réplicas dos outros brokers as partições que lhe cabem.

2.3. Sensores (produtores)

Os sensores são simulados por um programa em Python (`sensor.py`) empacotado em uma imagem Docker. Cada máquina da fábrica corresponde a um Deployment (`producer-maquina-1`, `-2` e `-3` na implantação inicial), o que permite dar a cada uma um ritmo próprio por variáveis de ambiente. Novas máquinas podem ser criadas durante a execução (ver Tabela 3), cada uma com seu próprio Deployment.

Cada máquina envia medições de quatro tipos (temperatura, vibração, energia e CO₂), cada tipo em seu próprio intervalo, igual ao intervalo base da máquina multiplicado por um fator configurável por tipo. Cada medição é enviada ao tópico `dados-sensores` como uma mensagem JSON contendo o identificador da máquina, o valor medido e um *timestamp*. Os valores são sorteados em faixas fixas por tipo de sensor,

iguais para todas as máquinas; o que diferencia as máquinas é apenas a frequência de envio, conforme a Tabela 2.

Máquina	Intervalo base	Temperatura	Vibração	Energia	CO2
maquina-1	1 s	1 s	1,5 s	2 s	5 s
maquina-2	0,5 s	0,5 s	1,75 s	1 s	2,5 s
maquina-3	2,5 s	7,5 s	3,75 s	10 s	12,5 s
maquina-4 em diante	1 s	1 s	1,5 s	2 s	5 s

Tabela 2: Intervalo entre medições de cada tipo, por máquina.

As mensagens são enviadas sem chave, e o cliente Kafka as distribui entre as doze partições do tópico. Com isso, a carga de uma mesma máquina se espalha por várias partições e, portanto, por vários consumidores, o que favorece o balanceamento. O custo dessa escolha é que o Kafka não garante a ordem entre medições de uma mesma máquina, já que ele só ordena mensagens dentro de uma partição.

2.4. Consumidores (processadores de dados)

Os consumidores são um programa Python (`processor.py`) executado em um Deployment com três réplicas. Quando o grupo é escalado para mais de três consumidores, os adicionais são criados como Deployments separados (`consumer-extra-1`, `consumer-extra-2`, ...), com a mesma imagem e o mesmo grupo, mas com um tempo de processamento simulado menor. Todos usam o mesmo identificador de grupo (`sensor-group`), de modo que o Kafka divide as partições do tópico entre elas: cada partição é lida por apenas um consumidor do grupo, e a carga é balanceada automaticamente. Se um consumidor entra ou sai do grupo, o Kafka redistribui as partições entre os que restam, num processo chamado de rebalanceio. Como o tópico tem doze partições, no máximo doze consumidores do grupo recebem partições; os que excedem esse número ficam ociosos.

Algumas configurações do consumidor determinam como ele se comporta nessas situações. Primeiro, a posição de leitura (*offset*) só é confirmada ao Kafka depois que a mensagem é processada, e não automaticamente: se um consumidor cai no meio do processamento, a mensagem é entregue de novo ao consumidor que assumir a partição, em vez de ser perdida. Segundo, o consumidor envia sinais de vida ao coordenador do grupo a cada 3 segundos, e é considerado falho se ficar 10 segundos sem enviá-los, o que limita o tempo até o rebalanceio após uma queda. Terceiro, ao receber o sinal de término do Kubernetes (por exemplo, ao ser removido ou quando o Deployment é reduzido), o consumidor encerra de forma controlada e sai do grupo imediatamente, sem esperar esse prazo. Por fim, cada leitura do Kafka traz no máximo 10 mensagens, o que mantém curto o intervalo entre consultas ao coordenador.

Para cada mensagem, o consumidor compara o valor medido com o limite do tipo de sensor. Os limites seguem um perfil (`strict`, `normal` ou `loose`), escolhido por variável de ambiente e que pode ser sobrescrito individualmente; no perfil `normal`, por exemplo, os limites são 80 para a temperatura, 3,5 para a vibração, 350 para a energia e 600 para o CO2. Cada leitura é gravada no banco, com uma marca indicando se ela disparou um alerta, e cada alerta gera também um evento de auditoria. Para simular um processamento analítico custoso e assim tornar visível o acúmulo de mensagens (*lag*) sob carga, o consumidor aguarda, a cada mensagem, um tempo sorteado entre valores mínimo e máximo configuráveis (0,2 a 0,5 segundo nos consumidores do Deployment principal e 0,1 a 0,2 segundo nos consumidores extras). Se a conexão com o banco cai, ela é refeita antes da próxima mensagem; se o processamento de uma mensagem falha, o erro é registrado e o consumidor segue para a próxima.

Além de registrar alertas isolados, os consumidores avaliam a frequência de alertas por máquina. Como as medições de uma máquina se espalham por várias partições, elas são processadas por consumidores diferentes; por isso o histórico de alertas não pode ficar na memória de cada consumidor e é mantido em um Redis compartilhado, que soma os alertas de uma mesma máquina independentemente de quem os leu. Quando uma máquina atinge o número máximo de alertas (oito) dentro da janela de análise (7 segundos na configuração atual), o consumidor publica no tópico `comandos-fabrica` um comando KILL com o identificador da máquina.

2.5. Controlador

O controlador é um programa Python (`controlador.py`) executado em um Deployment com duas réplicas, todas no mesmo grupo de consumo (`controlador-instancias-group`) e lendo o tópico `comandos-fabrica`. Como pertencem ao mesmo grupo, cada comando é entregue a apenas uma réplica, o que evita que duas tentem remover o mesmo pod. Ao receber um comando KILL, o controlador usa a API do Kubernetes para localizar os pods cujo rótulo `sensor-id` corresponde à máquina e removê-los; o Deployment do sensor então cria um novo pod no lugar, o que simula o reinício de uma máquina com defeito.

Pods não podem remover outros pods por padrão, então o controlador executa com uma ServiceAccount própria, associada por um RoleBinding a uma Role que permite apenas listar, consultar e deletar pods no namespace. Cada ação do controlador (comando recebido, pod removido, máquina não encontrada) é registrada na tabela de eventos.

2.6. Armazenamento e configuração

Os dados processados são guardados em um PostgreSQL executado como StatefulSet, com um volume persistente de 1 GiB. O consumidor cria duas tabelas: `leituras_sensores`, com uma linha por medição (máquina, tipo de sensor, valor, indicação de alerta e instante), e `event_table`, com os eventos de auditoria de todos

os componentes (início, envio de dados, alertas, comandos e remoções de pods). O Adminer, uma interface web, permite consultar essas tabelas.

Nenhum valor de configuração está fixo nas imagens. Endereços, nomes de tópicos, limites, intervalos e tempos vêm de variáveis de ambiente, definidas em ConfigMaps do Kubernetes e injetadas nos pods. A senha do banco, por ser um dado sensível, não fica em ConfigMap: é guardada em um Secret (`postgres-credentials`), um objeto do próprio cluster e não um arquivo, criado durante a instalação com uma senha aleatória gerada uma única vez e lida pelos pods por referência ao Secret. Assim, a senha não é versionada: cada instalação gera a sua.

3. INSTALAÇÃO E USO

3.1. Pré-requisitos

A aplicação foi preparada para uma máquina Debian com acesso à internet e a um usuário com permissão de `sudo`. A instalação usa o Docker para construir as imagens e o K3s, uma distribuição leve do Kubernetes que já inclui o `kubectl`. Todos os comandos abaixo são executados na raiz do repositório.

Os scripts da pasta `scripts/` estão versionados com permissão de execução, e o `make init` aplica novamente a todos eles.

3.2. Instalação

A instalação é automatizada pelo Makefile, em três etapas.

1. Instalar as dependências (Docker e K3s) com `make init`. O comando exige `sudo` e, se o Docker for instalado pela primeira vez, é preciso encerrar a sessão e entrar novamente para que o usuário passe a pertencer ao grupo `docker`.
2. Ligar o cluster local com `make start-cluster`.
3. Construir as imagens e implantar tudo com `make all`. Esse comando constrói as imagens do sensor, do consumidor e do controlador, as importa para o K3s, aplica os manifestos do Kafka (ConfigMaps, Services, StatefulSet e Job de criação dos tópicos) e aplica os da aplicação (Secret com a senha do banco, ConfigMaps, PostgreSQL, Redis, Adminer, controlador, sensores e consumidores).

A conferência é feita com `make status`, que lista pods, serviços e StatefulSets. Após um ou dois minutos, todos os pods devem estar em estado `Running`, e o pod do Job de criação dos tópicos em `Completed`. Para obter logs detalhados das bibliotecas, a implantação pode ser feita com `make all DEBUG=1`, que ativa o modo de depuração nos ConfigMaps.

3.3. Operação

A Tabela 3 reúne os principais comandos de operação.

Comando	O que faz
<code>make status</code>	Lista pods, serviços e StatefulSets
<code>make logs-producer, make logs-consumer, make logs-controlador</code>	Acompanha os logs dos sensores, consumidores e controlador
<code>make kafka-lag</code>	Mostra, por partição, o consumidor responsável e o <i>lag</i> do grupo <code>sensor-group</code> , consultando qualquer broker em execução
<code>make db-shell</code>	Abre o <code>psql</code> dentro do pod do PostgreSQL
<code>make db-ui</code>	Expõe o Adminer na porta 8080, já autenticado no banco
<code>make db-password</code>	Exibe a senha do banco, lida do Secret
<code>make scale-producers MACHINES=N</code>	Ajusta o número de máquinas para N, criando os Deployments que faltam e removendo os excedentes (padrão: 8)
<code>make scale-machine MAQUINA=M P_REPLICAS=R</code>	Coloca R pods simultâneos na máquina M (padrão: máquina 1, 3 pods)
<code>make scale-consumers C_REPLICAS=N</code>	Ajusta o número total de consumidores para N: até 3 no Deployment principal e o restante como consumidores extras (padrão: 12)
<code>make test-all</code>	Executa o roteiro interativo de testes
<code>make clean</code>	Remove os recursos declarados nos manifestos, as máquinas e os consumidores extras criados durante a execução
<code>make db-reset</code>	Apaga o volume do PostgreSQL, zerando o banco

Tabela 3: Principais comandos de operação do Makefile.

O comando `make clean` não apaga o volume do PostgreSQL nem o Secret com a senha: por isso, os dados e a senha continuam os mesmos depois de um `make clean` seguido de `make all`. Para zerar o banco, use `make db-reset`; para também trocar a senha, apague o Secret junto com o volume.

O Adminer é aberto com login automático. Por isso, a porta 8080 exposta por `make db-ui` dá acesso ao banco sem senha a quem alcançar a máquina pela rede, o que é aceitável apenas em ambiente de demonstração.

4. TESTES DE FALHA E ELASTICIDADE

Os testes são executados por scripts da pasta `scripts/`. Cada script salva a saída em um arquivo próprio na pasta `logs/`, organizado em seções (antes, falha e depois). Cada comando aparece seguido de sua saída; falhas e tempos limite também são registrados, sem interromper o teste. Os tempos de espera, os nomes dos deployments e o número de réplicas podem ser configurados por variáveis de ambiente. O roteiro interativo (`make test-all`) também salva a saída em `logs/interactive_*.log`.

4.1. Falha de um broker Kafka

O script `test_broker_failover.sh` remove à força o primeiro pod do StatefulSet do Kafka, simulando uma falha abrupta. Antes e depois, registra líderes, réplicas e ISR de cada partição, o *lag* do grupo, a quantidade de leituras no banco e os logs dos consumidores. As consultas posteriores são feitas por meio de um broker sobrevivente. A continuidade do processamento é indicada pelo aumento das leituras no banco; os logs dos produtores não são evidência suficiente, pois o envio do cliente Kafka é assíncrono. Por padrão, o script aguarda 3 segundos após a falha antes de coletar o estado posterior. O resultado está descrito no capítulo de resultados, e o registro completo está em `logs/failover_broker_20260926_180852.log`.

4.2. Falha de um consumidor e rebalanceio

O script `test_consumer_rebalance.sh` remove um pod consumidor e registra a atribuição das partições antes e depois da falha, além dos logs dos consumidores nos segundos seguintes. Por padrão, a consulta posterior é feita 2 segundos após a remoção. O rebalanceio é demonstrado pela mudança do consumidor responsável pelas partições do pod removido e pelas mensagens de nova geração do grupo nos logs dos consumidores. O registro completo está em `logs/rebalanceio_consumidor_20260926_180813.log`.

4.3. Elasticidade

O script `test_elasticity.sh` demonstra a elasticidade em duas fases: primeiro amplia a produção (mais pods de uma mesma máquina) e, em seguida, aumenta o número de consumidores do Deployment principal, coletando amostras periódicas do *lag* em cada fase. Como as mensagens não têm chave, a carga dos produtores é distribuída entre as doze partições; o paralelismo útil do grupo também é limitado a esse número. Esse script não foi executado nos ensaios registrados; a elasticidade foi demonstrada pelo roteiro interativo, descrito a seguir.

O roteiro interativo (`make test-all`) percorre as mesmas situações passo a passo, com painéis de monitoramento atualizados na tela, e grava toda a saída em `logs/interactive_*.log`. Nele, a elasticidade é demonstrada de outra forma: em vez de mais pods de uma mesma máquina, o número de máquinas passa de três para oito, com cinco amostras do *lag*; em seguida, o grupo é escalado para 12 consumidores (os três do Deployment principal e nove extras), com mais cinco amostras. Depois, o roteiro executa os testes de rebalanceio e de failover e, ao final, devolve o sistema a três máquinas e três consumidores.

5. RESULTADOS

5.1. O que funcionou

Os testes foram executados em 26/09/2026, com o tópico dados-sensores configurado com 12 partições e fator de replicação 3, conforme os manifestos. No failover, antes da falha as lideranças estavam divididas igualmente entre os três brokers (quatro partições cada) e todas as partições tinham os três brokers em sincronia (ISR 0,1,2). Após a remoção de kafka-0, as 12 partições continuaram com líder, agora distribuídas entre kafka-1 (sete) e kafka-2 (cinco), e o ISR passou a 1,2. As leituras gravadas no PostgreSQL aumentaram de 56.573 para 56.706 nos cerca de 8 segundos entre as duas consultas, indicando continuidade do processamento, e o StatefulSet recriou o pod kafka-0. O registro está em `logs/failover_broker_20260926_180852.log`.

No teste de consumidor, o grupo tinha 12 membros (três do Deployment principal e nove extras), um por partição. O pod removido lia a partição 1; na consulta posterior, essa partição aparece atribuída a outro consumidor, que passou a ler também a partição 0, e nenhuma das 12 partições ficou sem consumidor. O Deployment criou um pod substituto, e os logs dos consumidores mostram o grupo concluindo uma nova geração (geração 9) cerca de 4 segundos após a remoção, já com o substituto recebendo uma partição (a partição 4). O registro está em `logs/rebalanco_consumidor_20260926_180813.log`.

O roteiro interativo evidenciou o efeito da elasticidade. Com três máquinas e três consumidores, o *lag* total do grupo era de 8 mensagens. Ao passar para oito máquinas, as cinco amostras seguintes mostraram o *lag* crescendo continuamente: 20, 84, 145, 222 e 290 mensagens, ou seja, a produção superou a capacidade dos três consumidores. Após o escalonamento para 12 consumidores, a primeira amostra ainda registrou 361 mensagens, e as seguintes caíram para 84, 30, 13 e 7. Esses dados estão em `logs/interactive_20260926_180644.log`.

5.2. O que não funcionou

No teste de rebalanceio, a consulta à tabela do grupo, feita 2 segundos após a remoção, ainda retornou o aviso `Consumer group 'sensor-group' is rebalancing`, e o pod substituto aparecia em `Init:0/1`. A conclusão do rebalanceio só aparece nos logs dos consumidores, e não em uma consulta posterior ao grupo; um tempo de espera maior no script permitiria registrar também a tabela final com o grupo estável.

Na elasticidade, o *lag* continuou crescendo brevemente depois de subir o número de consumidores: a primeira amostra após o escalonamento foi de 361, acima das 290 mensagens anteriores. A queda só aparece nas amostras seguintes, o que evidencia o custo de inicialização dos pods e do rebalanceio; adicionar réplicas não elimina instantaneamente uma fila já acumulada. Além disso, os nove consumidores extras usam um tempo de processamento simulado menor (0,1 a 0,2 segundo, contra 0,2 a 0,5 no Deployment principal), então a redução do *lag* resulta tanto do aumento

do número de consumidores quanto do processamento mais rápido de cada um, e o ensaio não separa os dois efeitos. Como o tópico tem 12 partições, consumidores além desse limite não receberiam partições. O script `test_elasticity.sh`, que gera um log dedicado de elasticidade, não foi executado nos ensaios registrados.

Há também limitações da configuração, não falhas específicas observadas nesses ensaios: os brokers Kafka não usam volumes persistentes; o Job de inicialização apaga e recria os tópicos a cada implantação; as mensagens dos sensores não têm chave, então não há garantia de ordenação por máquina; e o Adminer está configurado com acesso automático, sem autenticação própria. O teste de failover mostra que as réplicas mantiveram o serviço durante a janela medida, mas não prova sozinho ausência de perda de mensagens em qualquer cenário de falha.

6. CONCLUSÃO

Os testes demonstraram, no ambiente executado, a continuidade de leitura após a queda de um broker, a redistribuição de partições após a remoção de um consumidor e a redução do *lag* depois do aumento de consumidores. O experimento de carga também mostrou que a capacidade de processamento pode ficar temporariamente abaixo da taxa de produção e que o escalonamento leva algum tempo para surtir efeito. Os resultados são coerentes com uma arquitetura que combina replicação do Kafka, grupos de consumo e recuperação de pods pelo Kubernetes.

Como próximos passos, recomenda-se ampliar o tempo de espera do teste de rebalanceamento e só registrar o resultado depois de confirmar que o grupo está estável e que o pod substituto está pronto. Também são melhorias relevantes persistir os dados dos brokers, proteger o acesso ao Adminer e usar a chave da máquina nas mensagens caso a ordem por máquina seja necessária. Por fim, testes de carga repetidos e com métricas de duração e volume permitiriam comparar quantitativamente diferentes configurações de produtores e consumidores.

REFERÊNCIAS

APACHE SOFTWARE FOUNDATION. **Apache Kafka Documentation**. Disponível em: <<https://kafka.apache.org/documentation/>>. Acesso em: 25 set. 2026.

THE KUBERNETES AUTHORS. **Kubernetes Documentation**. Disponível em: <<https://kubernetes.io/docs/>>. Acesso em: 25 set. 2026.

APÊNDICE A – Logs de execução

Os trechos abaixo resumem as evidências coletadas. Os arquivos completos, com as tabelas por partição e os logs dos pods, estão na pasta `logs/`.

AA Failover do broker

No teste de failover, o tópico tinha 12 partições, fator de replicação 3 e ISR 0,1,2 antes da falha. Após a remoção de `kafka-0`, todas as partições mantiveram um líder e o ISR passou a 1,2. O `StatefulSet` recriou o pod, e a contagem de leituras no banco aumentou de 56.573 para 56.706. O log completo está em `logs/failover_broker_20260926_180852.log`.

AB Rebalanceamento de consumidores

No teste de rebalanceamento, a partição 1, lida pelo consumidor removido, passou a outro consumidor, e nenhuma partição ficou sem consumidor. Os logs dos consumidores mostram a geração 9 do grupo concluída cerca de 4 segundos após a remoção, com o pod substituto recebendo a partição 4. A consulta ao grupo feita 2 segundos após a remoção ainda indicava rebalanceamento em andamento. O log completo está em `logs/rebalanco_consumidor_20260926_180813.log`.

AC Elasticidade

No roteiro interativo, o *lag* total do grupo foi de 8 mensagens no início; após a ampliação para oito máquinas, as amostras foram 20, 84, 145, 222 e 290. Com o grupo ampliado para 12 consumidores, as amostras foram 361, 84, 30, 13 e 7 mensagens, mostrando um pico inicial seguido pela redução da fila. O registro completo está em `logs/interactive_20260926_180644.log`.